Fábrica de Software

Profª. Drª. Daniela Bagatini

Bem-vindos!

Fábrica de Software

Competências a desenvolver

Desenvolver raciocínio lógico.

- Reconhecer a importância do **pensamento** computacional e lógico no cotidiano e aplicá-lo em circunstâncias apropriadas e em domínios diversos.

1 Raciocínio Lógico

Pensamento e organização

2 Análise e Concepção

Solução
Lógica e técnica

Aprofundar a capacidade de análise e concepção de algoritmos.

- Explorar o **raciocínio abstrato** como ferramenta para a resolução de problemas de forma computacional.

3 Aplicação e Programação

Lógica e técnica

Compreender e aplicar o conceito de modularização no desenvolvimento de programas.

- Resolver problemas usando ambientes e linguagens de programação, através da seleção e criação de algoritmos apropriados, bem como implementação da solução.

Fábrica de Software

Organização, atividades e avaliação

1ª semana — Boas-vindas, apresentação e introdução

Encontro **Tópicos de trabalho** (ver plano de ensino na sala virtual)

PYTHON

- Modularização (declaração, parâmetros, bibliotecas) e recursividade
- Ponteiros e alocação dinâmica
- Métodos de pesquisa
- Métodos de ordenação
- Arquivos
- Discovery ★

Dinâmica para cada tema:

1º encontro – Aula preparatória e pré-aula
2º encontro - Aula mediada
3º encontro – Pós-aula

Encontros remotos: ver cronograma no plano de ensino

JAVA

- Introdução a OO (classes, objetos e métodos, variáveis e estruturas)
- Encapsulamento (Modificadores de acesso, construtores, getters e setters)
- Atributos de classe (static), sobrecarga de métodos, múltiplos construtores
- Herança (reuso, reescrita, sobrecarga de métodos e múltiplos construtores)
- Arrays e Listas
- Polimorfismo (de referência, de sobrecarga e de sobreescrita)
- Classes e métodos abstratos e Interfaces
- Associação, composição e agregação
- Tipos enumerados

Dinâmica: Temas por aula

Encontros remotos: ver cronograma no plano de ensino

Avaliações de andamento:

3 pontos para Python (Desafios) + 3 pontos para Java, totalizando 6 pontos

Projeto (4 pontos), sendo 1 ponto Pitch + 3 pontos PFS: desenvolvimento + entrega + apresentação /defesa

Junho	Desenvolvimento Projeto de Fábrica de Software
Julho	Defesa do Projeto de Fábrica de Software (PFS)
Julho	Fechamento e entrega de notas
Julho	Exame (ver plano de ensino)

Frequência pelos encontros presenciais + remotos: participação, entregas e apresentações/defesas.
Contato pelo fórum de dúvidas (sala virtual) + aulas remotas + aulas presenciais.

Fábrica de Software

Sala de Aula Invertida!

STEP 1

Aula preparatória

Mobilização para a aprendizagem
- Engajamento experiencial e apresentação roteiro de estudos -

STEP 2

Pré-Aula
(estudo individual)

Construção do conhecimento – exploração conceitual -

STEP 3

Aula mediada

Síntese da aprendizagem – conectar conceitos / produção de sentido -

STEP 4

Pós-Aula

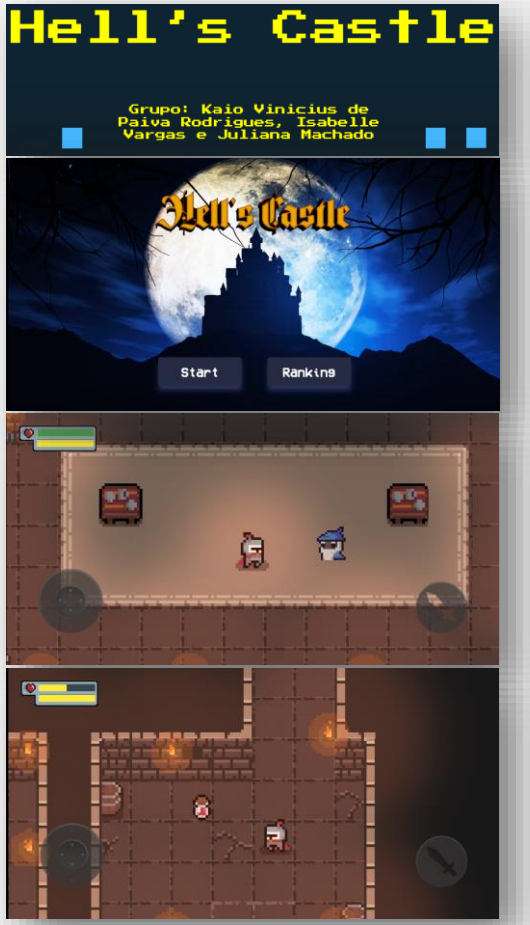
Apresentação dos resultados

Fábrica de Software

Sala virtual e Plano de ensino

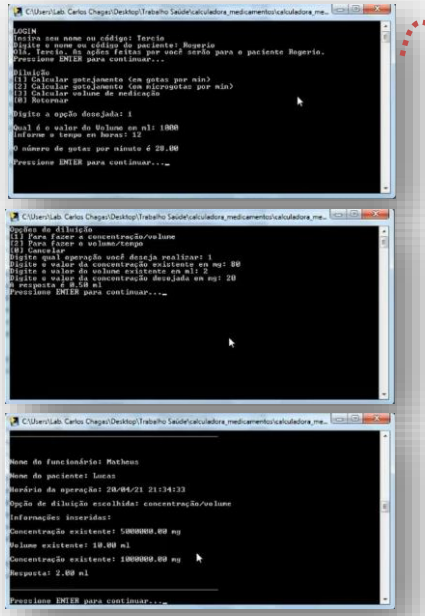
Fábrica de Software

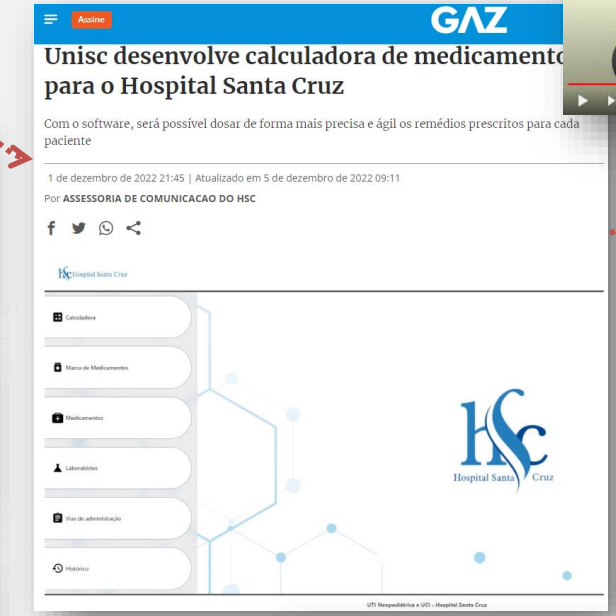
Exemplos de PFS



Jogo RPG

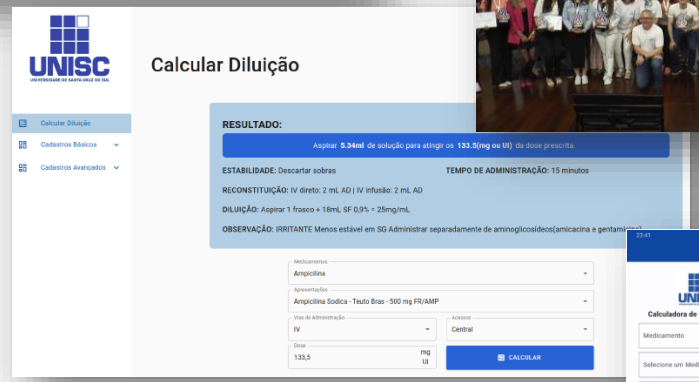
Aplicar conhecimentos de Fábrica de Software: pesquisa, ordenação, hash.
Ir além: integração do back-end em Python com o Flutter (Kaio, Isabelle e Juliana).





Calculadora de medicamentos

Aplicar conhecimentos de Fábrica de Software: pesquisa, ordenação, arquivos.
Ir além: cálculo de gotejamento (Turma de 2021-1).



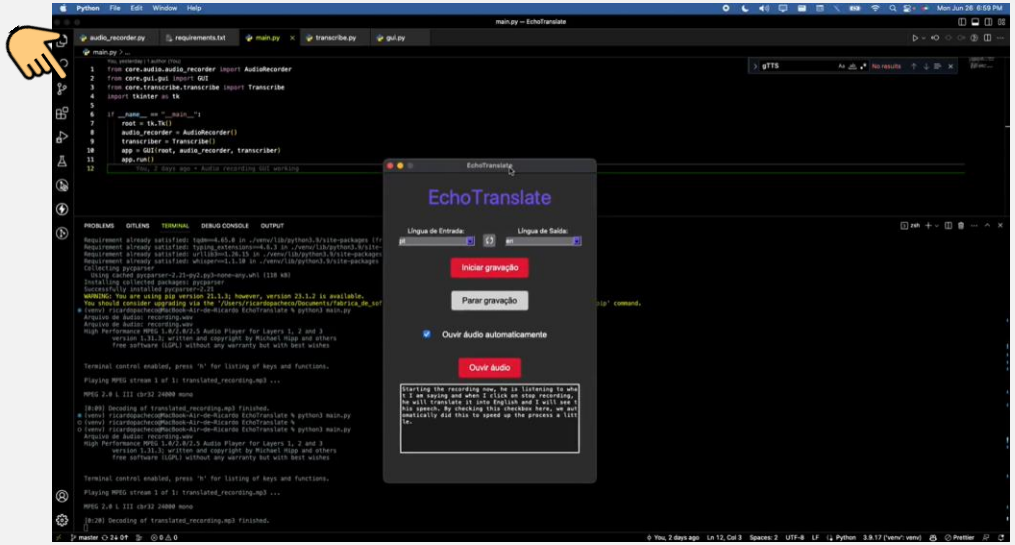
Jean Kistenmacher

Fábrica de Software

Exemplos de PFS

EchoTranslate

Gabriel Bartholomay, Ricardo Pacheco, Vinicius Mohr e Wilhelm Bock

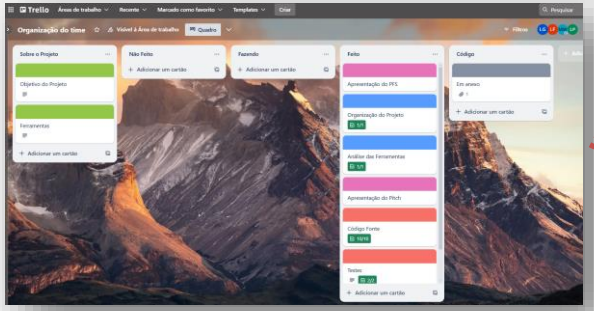


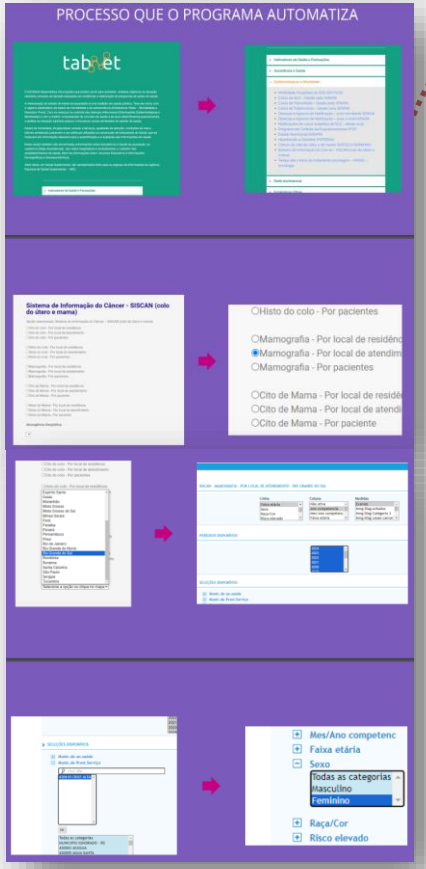
EchoTranslate

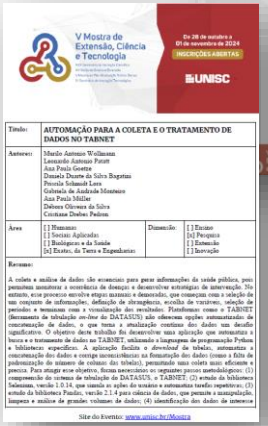
Facilitar a comunicação entre pessoas que falam diferentes idiomas através do uso da tecnologia, traduzindo áudio em tempo real e convertendo a fala em outra língua (Gabriel, Ricardo, Vinicius e Wilhelm).

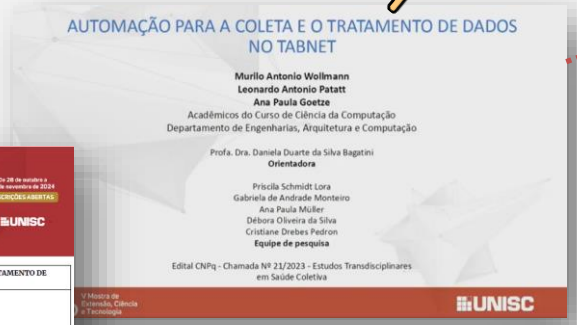
Fábrica de Software

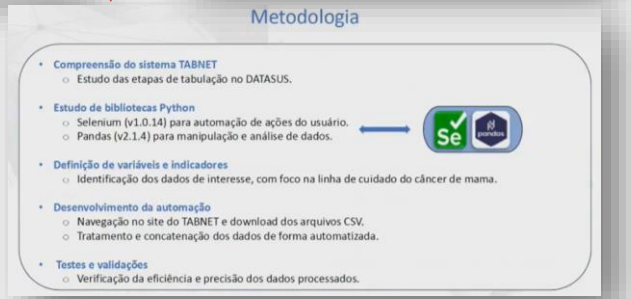
Exemplos de PFS

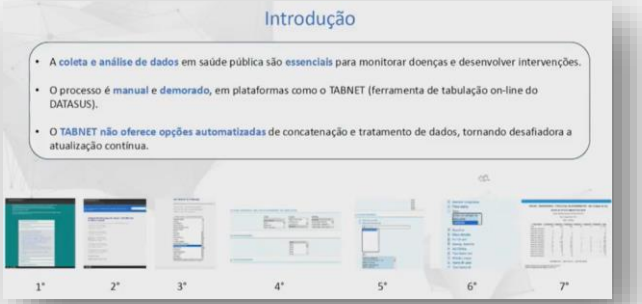


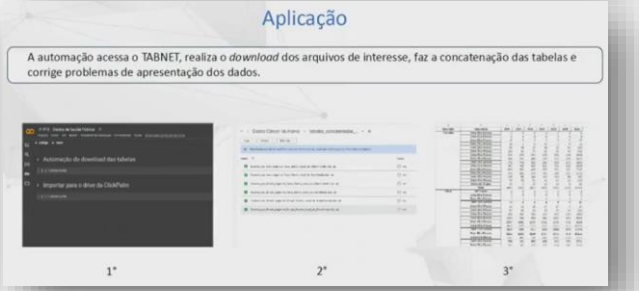










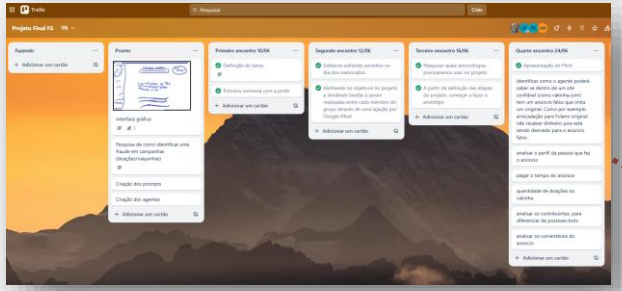


Automação da busca e tratamento de dados de saúde pública

Aplicar conhecimentos de Fábrica de Software: pesquisa, ordenação, arquivos.
Ir além: automação de tarefas (Turma de 2024-1).

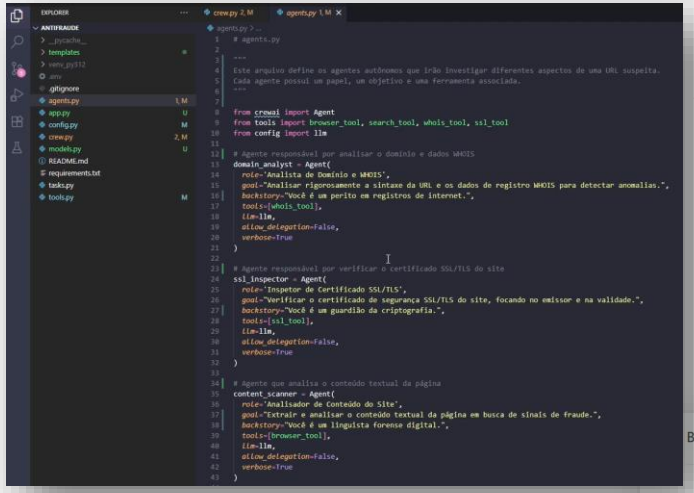
Fábrica de Software

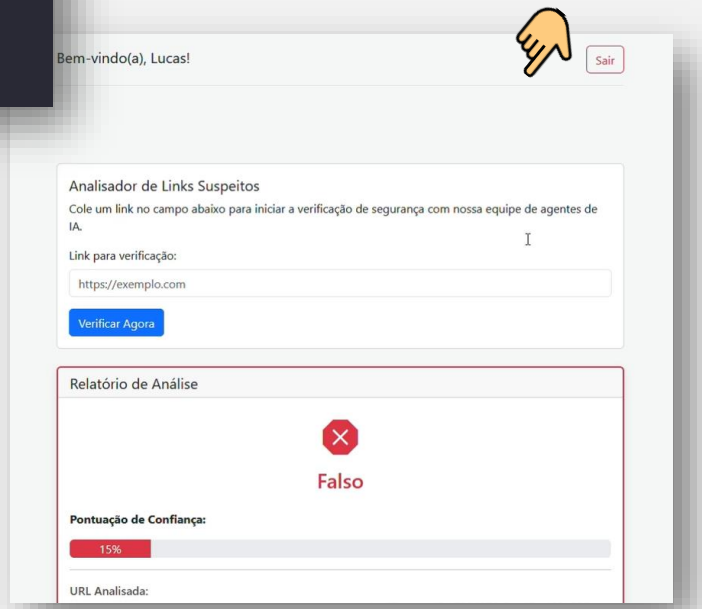
Exemplos de PFS



AntiFraude

Aplicar conhecimentos de Fábrica de Software: pesquisa, arquivos.
Ir além: agentes autônomos (Turma 1e 2 de 2025-1).





Fábrica de Software

O que vamos estudar agora?

Modularização (declaração, parâmetros, bibliotecas) e recursividade

Objetivos de aprendizagem:

Compreender as principais primitivas de programação, organizar os passos de resolução e aplicar na solução de problemas iniciais utilizando modularização.

Aula preparatória:

Modularização (declaração, parâmetros, bibliotecas)

Pré-aula (Estudos individuais):

Investigar e testar diferentes IDEs e escolher uma.

Interpretar, analisar e resolver os problemas propostos no Beecrowd.

Material de referência:

Modularização, dados de entrada e saída em Python

Introdução à lógica de programação e ambiente Python

Tipos de dados e variáveis (utilizando a linguagem Python)

Comando condicional if (estruturas simples e compostas) — Python

Comando while, variáveis contadoras e acumuladoras (utilizando a linguagem Python)

Comando for, variáveis contadoras e acumuladoras (utilizando a linguagem Python)

Fábrica de Software

Primeiro exemplo

STEP 1

Entenda o problema e informações fornecidas

Por meio de uma entrevista, queremos conhecer qual o Sistema Operacional Mobile mais utilizado por nossos clientes!

Nossa atividade é desenvolver um algoritmo para coletar dados sobre: quantos clientes usam o sistema operacional iOS e quantos usam o Android; quantas pessoas foram entrevistadas; e, qual a idade média dos entrevistados.

Qual o Sistema Operacional Mobile mais utilizado!

Contar quantidade de IOS informados! ✓

Contar quantidade de Android informados! ✓

Calcular a idade média dos entrevistados! ✓

Fábrica de Software

Primeiro exemplo

Organização da solução!

STEP 2

Identifique objetivo, entradas, saídas e casos de teste

OBJETIVO

Identificar o sistema operacional (iOS ou Android).

ENTRADAS

Sistema operacional, idade, nº de entrevistados

SAÍDAS

nº de iOS, nº de Android, idade média dos entrevistados

CASOS DE TESTE

Caso de teste N

Caso de teste 2

Caso de teste 1

Entrada	Saída
Número de entrevistados: 3 Digite idade: 30 Digite SO: Android Digite idade: 20 Digite SO: iOS Digite idade: 25 Digite SO: Android	Nº iOS 1 Nº Android 2 Média das idades: 25

Fábrica de Software

Primeiro exemplo

Algoritmo!

De acordo com Cormen *et al.* (2012), um **algoritmo** é uma sequência de instruções, escritas de forma clara, ordenada e finita, e elaborada para realizar uma determinada tarefa ou resolver um problema.

STEP 3

Elabore o algoritmo (textual ou diagrama)

Informar número de entrevistados

Para ir de 1 até número de entrevistados

- Informar idade
- Informar o sistema operacional [iOS ou Android]
- Acumular a idade
- Se sistema for iOS, contar mais um iOS
  - Caso contrário, contar mais um Android

Calcular idade média

Mostrar número de iOS e Android, idade média

1º passo: descrever de forma clara e objetiva.

2º passo: relacionar todos os itens necessários.

3º passo: ordenar as ações para resolver o problema.

4º passo: avaliar se o objetivo foi alcançado através das sequências de instruções.

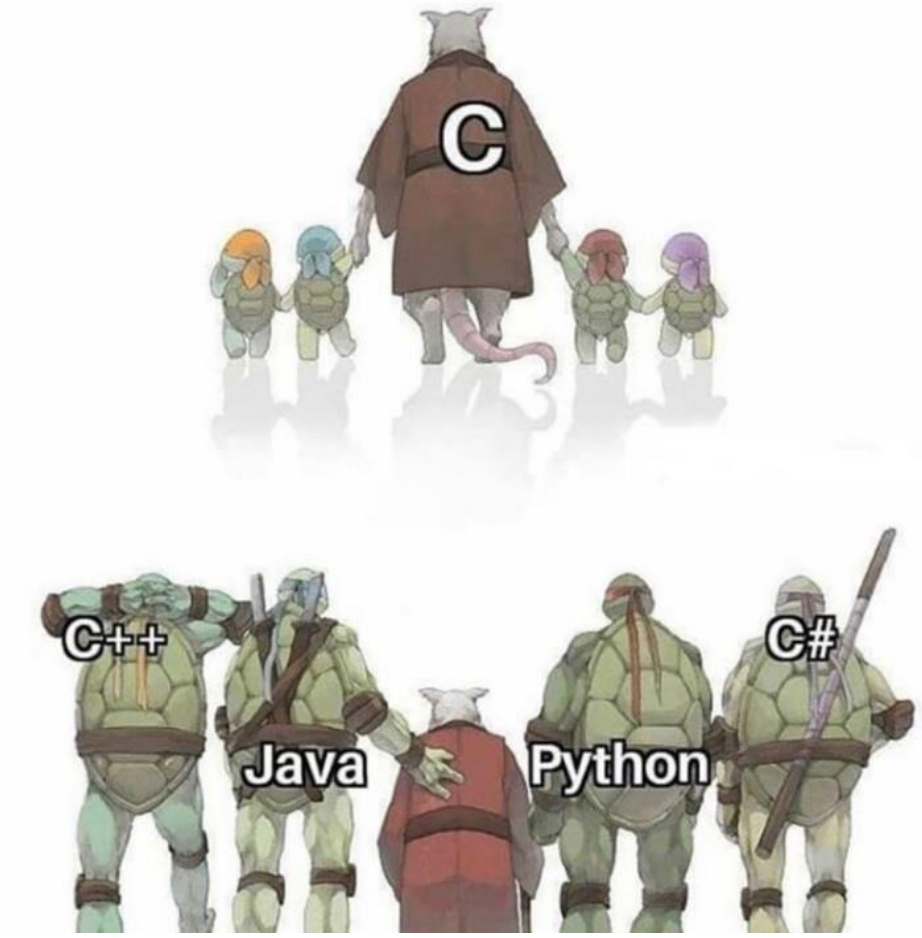
Fábrica de Software

Primeiro exemplo

Programa!

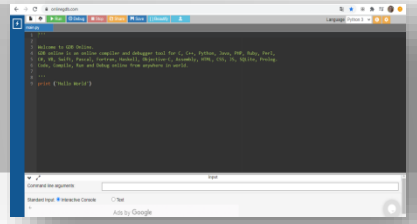
STEP 4

Desenvolva o programa e promova melhorias



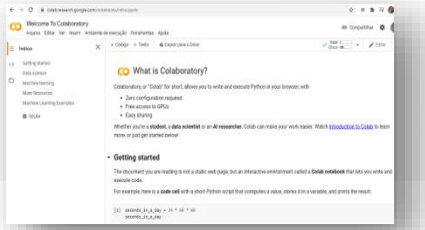
IDEs para Python

OnlineGDB



https://www.onlinegdb.com/
https://repl.it/languages/python3

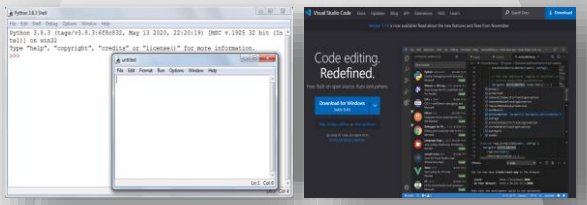
Colab



https://colab.research.google.com/notebooks/intro.ipynb

Utilizaremos a versão Python 3.x

IDLE
VS Code



https://www.python.org/
https://code.visualstudio.com/

Fonte: https://www.tiobe.com/tiobe-index/

Em Fábrica de Software utilizaremos o Colab,
no entanto você pode escolher o seu ambiente de preferência para trabalho.

Fábrica de Software

Primeiro exemplo

STEP 4

Desenvolva o programa e promova melhorias

Programa em Python! Esta é a versão 1.0, no Colab você também encontra o código aprimorado (versão 2.0).

```
def entrevistados():  # função entrevistados()
  contadorIOS = 0  # contadores
  contadorAndroid = 0
  acumuladorIdade = 0  # acumulador

  numeroEntrevistados = int(input("Número de entrevistados: "))  # entrada de dados inteira

  for _ in range(numeroEntrevistados):  # número de repetições para o código, o range iterar sobre
sequências numéricas, no caso de 0 até nEntrevistados (exclusive)
    idade = int(input("Digite idade: "))  # ler dado de entrada inteiro
    so = input("Digite SO [iOS | Android]: ")  # ler dado de entrada (por padrão do tipo string)
    acumuladorIdade += idade  # acumular idades (igual a: aIdade = aIdade + 1)

    if so == "iOS":  # verifica caminhos possíveis...
      contadorIOS += 1  # se iOS verdadeiro, contar iOS
    else:
      contadorAndroid += 1  # senão, contar Android

  mediaIdade = acumuladorIdade / numeroEntrevistados  # calcular a média das idades
  print("\nN° iOS: {} N° Android: {} - Média das idades: {}".format(contadorIOS, contadorAndroid,
mediaIdade)) # escrever em tela dados de saída formatados

entrevistados()
```

Fábrica de Software

Primeiro exemplo

ALGORITMO

Informar número de entrevistados

Para ir de 1 até número de entrevistados

- Informar idade
- Informar o sistema operacional [iOS ou Android]
- Acumular a idade
- Se sistema for iOS, contar mais um iOS
  - Caso contrário, contar mais um Android

Calcular idade média

Mostrar número de iOS e Android, idade média

PROGRAMA

```
def entrevistados():
  contadorIOS = 0
  contadorAndroid = 0
  acumuladorIdade = 0

  numeroEntrevistados = int(input("Número entrevistados: "))
  for _ in range(numeroEntrevistados):
    idade = int(input("Digite idade: "))
    so = input("Digite SO [iOS | Android]: ")
    acumuladorIdade += idade

    if so == "iOS":
      contadorIOS += 1
    else:
      contadorAndroid += 1

  mediaIdade = acumuladorIdade / numeroEntrevistados
  print("\nN° iOS: {} N° Android: {} - Média das idades:
{}".format(contadorIOS, contadorAndroid, mediaIdade))

entrevistados()
```

atini

Fábrica de Software

Primeiro exemplo

ALGORITMO

Informar número de entrevistados

Para ir de 1 até número de entrevistados

- Informar idade
- Informar o sistema operacional [iOS ou Android]
- Acumular a idade
- Se sistema for iOS, contar mais um iOS
  - Caso contrário, contar mais um Android

Calcular idade média

Mostrar número de iOS e Android, idade média

PROGRAMA

```
def entrevistados():
  contadorIOS = 0
  contadorAndroid = 0
  acumuladorIdade = 0

  numeroEntrevistados = int(input("Número entrevistados: "))
  for _ in range(numeroEntrevistados):
    idade = int(input("Digite idade: "))
    so = input("Digite SO [iOS | Android]: ")
    acumuladorIdade += idade

    if so == "iOS":
      contadorIOS += 1
    else:
      contadorAndroid += 1

  mediaIdade = acumuladorIdade / numeroEntrevistados
  print("\nN° iOS: {} N° Android: {} - Média das idades:
{}".format(contadorIOS, contadorAndroid, mediaIdade))

entrevistados()
```

Fábrica de Software

Primeiro exemplo

ALGORITMO

Informar número de entrevistados

Para ir de 1 até número de entrevistados

- Informar idade
- Informar o sistema operacional [iOS ou Android]
- Acumular a idade
- Se sistema for iOS, contar mais um iOS
  - Caso contrário, contar mais um Android

Calcular idade média

Mostrar número de iOS e Android, idade média

PROGRAMA

```
def entrevistados():
  contadorIOS = 0
  contadorAndroid = 0
  acumuladorIdade = 0

  numeroEntrevistados = int(input("Número entrevistados: "))
  for _ in range(numeroEntrevistados):
    idade = int(input("Digite idade: "))
    so = input("Digite SO [iOS | Android]: ")
    acumuladorIdade += idade

    if so == "iOS":
      contadorIOS += 1
    else:
      contadorAndroid += 1

  mediaIdade = acumuladorIdade / numeroEntrevistados
  print("\nN° iOS: {} N° Android: {} - Média das idades:
{}".format(contadorIOS, contadorAndroid, mediaIdade))

entrevistados()
```

Fábrica de Software

Primeiro exemplo

ALGORITMO

Informar número de entrevistados

Para ir de 1 até número de entrevistados

- Informar idade
- Informar o sistema operacional [iOS ou Android]
- Acumular a idade
- Se sistema for iOS, contar mais um iOS
  - Caso contrário, contar mais um Android

Calcular idade média

Mostrar número de iOS e Android, idade média

PROGRAMA

```
def entrevistados():
  contadorIOS = 0
  contadorAndroid = 0
  acumuladorIdade = 0

  numeroEntrevistados = int(input("Número entrevistados: "))
  for _ in range(numeroEntrevistados):
    idade = int(input("Digite idade: "))
    so = input("Digite SO [iOS | Android]: ")
    acumuladorIdade += idade

    if so == "iOS":
      contadorIOS += 1
    else:
      contadorAndroid += 1

  mediaIdade = acumuladorIdade / numeroEntrevistados
  print("\nN° iOS: {} N° Android: {} - Média das idades:
{}".format(contadorIOS, contadorAndroid, mediaIdade))

entrevistados()
```

Fábrica de Software

Primeiro exemplo

PROGRAMA

```
def entrevistados():
  contadorIOS = 0
  contadorAndroid = 0
  acumuladorIdade = 0

  numeroEntrevistados = int(input("Número entrevistados: "))
  for _ in range(numeroEntrevistados):
    idade = int(input("Digite idade: "))
    so = input("Digite SO [iOS | Android]: ")
    acumuladorIdade += idade

    if so == "iOS":
      contadorIOS += 1
    else:
      contadorAndroid += 1

  mediaIdade = acumuladorIdade / numeroEntrevistados
  print("\nN° iOS: {} N° Android: {} - Média das idades:
{}".format(contadorIOS, contadorAndroid, mediaIdade))

entrevistados()
```

Perceba que uma variável permite armazenar um dado em memória RAM e ter acesso a ele por meio do nome da variável.

Você sabe por que os programadores atribuem nomes às variáveis?

Variável

Para armazenar em memória um valor atribuído (=) e, posteriormente, utilizar este valor sempre que necessário. Por exemplo: **contadorIOS**.

Fábrica de Software

Primeiro exemplo

PROGRAMA

```
def entrevistados():
  contadorIOS = 0
  contadorAndroid = 0
  acumuladorIdade = 0

  numeroEntrevistados = int(input("Número entrevistados: "))
  for _ in range(numeroEntrevistados):
    idade = int(input("Digite idade: "))
    so = input("Digite SO [iOS | Android]: ")
    acumuladorIdade += idade

    if so == "iOS":
      contadorIOS += 1
    else:
      contadorAndroid += 1

  mediaIdade = acumuladorIdade / numeroEntrevistados
  print("\nN° iOS: {} N° Android: {} - Média das idades:
{}".format(contadorIOS, contadorAndroid, mediaIdade))

entrevistados()
```

Durante a execução do programa, o Sistema Operacional vai se deparar com aquele nome (etiqueta) e vai buscar por ele na memória principal. Quando encontrar o nome, o valor que atribuímos a ele estará armazenado no espaço que foi reservado para a variável, ou seja, é pelo nome da variável que o Sistema Operacional recupera o último valor armazenado!

RAM – Memória principal

X	10
1027	
1028	
1029	
1030	

Fábrica de Software

Primeiro exemplo

PROGRAMA

```
def entrevistados():
  contadorIOS = 0
  contadorAndroid = 0
  acumuladorIdade = 0

  numeroEntrevistados = int(input("Número entrevistados: "))
  for _ in range(numeroEntrevistados):
    idade = int(input("Digite idade: "))
    so = input("Digite SO [iOS | Android]: ")
    acumuladorIdade += idade

    if so == "iOS":
      contadorIOS += 1
    else:
      contadorAndroid += 1

  mediaIdade = acumuladorIdade / numeroEntrevistados
  print("\nN° iOS: {} N° Android: {} - Média das idades:
{}".format(contadorIOS, contadorAndroid, mediaIdade))

entrevistados()
```

O **tipo** indica a natureza dos dados armazenados em uma variável.
Por exemplo: sendo uma variável de tipo caractere, o Python saberá que não é possível realizar uma operação de cálculo com esta variável.

Tipos de dados

No programa você encontra três tipos de dados principais: inteiro, real e caractere. Por exemplo: cIOS é uma variável que recebe um valor inteiro; já so é uma variável que recebe um valor string (caracteres) e mIdade é uma varável que recebe um valor real.

Fábrica de Software

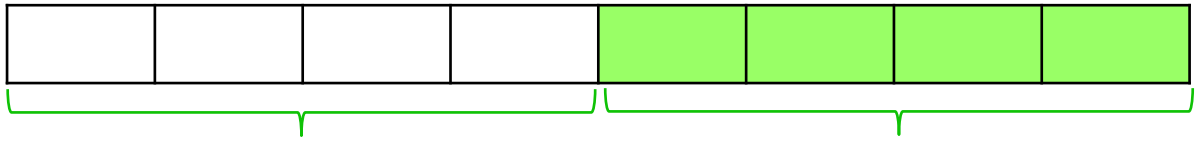
Primeiro exemplo

PROGRAMA

```
def entrevistados():
  contadorIOS = 0
  contadorAndroid = 0
  acumuladorIdade = 0

  numeroEntrevistados = int(input("Número entrevistados: "))
  for _ in range(numeroEntrevistados):
    idade = int(input("Digite idade: "))
    so = input("Digite SO [iOS | Android]: ")
    acumuladorIdade += idade

    if so == "iOS":
      contadorIOS += 1
    else:
      contadorAndroid += 1

  mediaIdade = acumuladorIdade / numeroEntrevistados
  print("\nN° iOS: {} N° Android: {} - Média das idades:
{}".format(contadorIOS, contadorAndroid, mediaIdade))

entrevistados()
```

Com a indicação do tipo do dado que será armazenado, o Sistema Operacional sabe quantos bytes de memória precisa reservar para guardar a informação.

E não é só isso: sabendo o tipo, o Sistema Operacional sabe quais operações ele pode fazer com aquele valor armazenado. Se o programador quiser dar uma de espertinho e, por exemplo, fazer uma multiplicação com caracteres, o Sistema Operacional não vai permitir.



Um dado do tipo inteiro, por exemplo, ocupa 4 bytes consecutivos na memória.

Fábrica de Software

Primeiro exemplo

PROGRAMA

```
def entrevistados():
  contadorIOS = 0
  contadorAndroid = 0
  acumuladorIdade = 0

  numeroEntrevistados = int(input("Número entrevistados: "))
  for _ in range(numeroEntrevistados):
    idade = int(input("Digite idade: "))
    so = input("Digite SO [iOS | Android]: ")
    acumuladorIdade += idade

    if so == "iOS":
      contadorIOS += 1
    else:
      contadorAndroid += 1

  mediaIdade = acumuladorIdade / numeroEntrevistados
  print("\nN° iOS: {} N° Android: {} - Média das idades:
{}".format(contadorIOS, contadorAndroid, mediaIdade))

entrevistados()
```

Tipos de dados

- Inteiro
- Real
- Caractere

Fábrica de Software

Primeiro exemplo

PROGRAMA

```
def entrevistados():
  contadorIOS = 0
  contadorAndroid = 0
  acumuladorIdade = 0

  numeroEntrevistados = int(input("Número entrevistados: "))
  for _ in range(numeroEntrevistados):
    idade = int(input("Digite idade: "))
    so = input("Digite SO [iOS | Android]: ")
    acumuladorIdade += idade

    if so == "iOS":
      contadorIOS += 1
    else:
      contadorAndroid += 1

  mediaIdade = acumuladorIdade / numeroEntrevistados
  print("\nN° iOS: {} N° Android: {} - Média das idades:
{}".format(contadorIOS, contadorAndroid, mediaIdade))

entrevistados()
```

Assim como a função criada pelo programador chamada **entrevistados()**, podemos observar outras existentes como **input()** e **print()**.

Função ()

Permite o reuso e a organização do código. Cada função realiza uma ação para a qual foi programada. Input lê dados de entrada, informados pelo usuário; print mostra dados em tela; entrevistados contabiliza o nº de sistemas, pessoas e idades.

Fábrica de Software

Primeiro exemplo

PROGRAMA

```
def entrevistados():
  contadorIOS = 0
  contadorAndroid = 0
  acumuladorIdade = 0

  numeroEntrevistados = int(input("Número entrevistados: "))
  for _ in range(numeroEntrevistados):
    idade = int(input("Digite idade: "))
    so = input("Digite SO [iOS | Android]: ")
    acumuladorIdade += idade

    if so == "iOS":
      contadorIOS += 1
    else:
      contadorAndroid += 1

  mediaIdade = acumuladorIdade / numeroEntrevistados
  print("\nN° iOS: {} N° Android: {} - Média das idades:
{}".format(contadorIOS, contadorAndroid, mediaIdade))

entrevistados()
```

Veja que toda informação conhecida do problema, é uma **entrada**. Por exemplo, a idade que será informada pelo usuário.
Já, toda informações que precisa ser fornecida pelo programa como resultado é uma **saída**. Por exemplo: mIdades (média das idades).

Entrada e saída

No programa você também encontra comandos de entrada como o input, e também, comandos de saída como o print.

Fábrica de Software

Primeiro exemplo

PROGRAMA

```
def entrevistados():
  contadorIOS = 0
  contadorAndroid = 0
  acumuladorIdade = 0

  numeroEntrevistados = int(input("Número entrevistados: "))
  for _ in range(numeroEntrevistados):
    idade = int(input("Digite idade: "))
    so = input("Digite SO [iOS | Android]: ")
    acumuladorIdade += idade

    if so == "iOS":
      contadorIOS += 1
    else:
      contadorAndroid += 1

  mediaIdade = acumuladorIdade / numeroEntrevistados
  print("\nN° iOS: {} N° Android: {} - Média das idades:
{}".format(contadorIOS, contadorAndroid, mediaIdade))

entrevistados()
```

Outras estruturas para loop utilizam uma condição como critério de repetição, este é o caso do **while**, por exemplo:

```
dia = 1
while dia <= 31:  #se a condição é verdadeira, executa os comandos
  idade= int(input("Digite idade:"))
  ...
  dia += 1  #ou seja, i = i + 1
```

Estrutura de controle – loop

No programa ao lado foi utilizado o for para repetir um bloco de código por um número determinado de entrevistas (informado).

Fábrica de Software

Primeiro exemplo

PROGRAMA

```
def entrevistados():
  contadorIOS = 0
  contadorAndroid = 0
  acumuladorIdade = 0

  numeroEntrevistados = int(input("Número entrevistados: "))
  for _ in range(numeroEntrevistados):
    idade = int(input("Digite idade: "))
    so = input("Digite SO [iOS | Android]: ")
    acumuladorIdade += idade

    if so == "iOS":
      contadorIOS += 1
    else:
      contadorAndroid += 1

  mediaIdade = acumuladorIdade / numeroEntrevistados
  print("\nN° iOS: {} N° Android: {} - Média das idades:
{}".format(contadorIOS, contadorAndroid, mediaIdade))

entrevistados()
```

se a condição for verdadeira

se a condição for falsa

O controle if-else oferece duas alternativas de execução de comandos. Se uma condição for verdadeira é escolhida a alternativa logo após o if, caso contrário (falso), segue-se um segundo caminho. É possível encadear vários if's (um if dentro de outro).

Estrutura de controle – decisão

Permite desviar o fluxo de execução de um programa de acordo com o resultado de testes (condição), por exemplo, de so for igual (==) a "iOS", então será contado + 1 em contadorIOS, senão será contado + 1 em contadorAndroid.

Fábrica de Software

Primeiro exemplo

PROGRAMA

```
def entrevistados():
  contadorIOS = 0
  contadorAndroid = 0
  acumuladorIdade = 0

  numeroEntrevistados = int(input("Número entrevistados: "))
  for _ in range(numeroEntrevistados):
    idade = int(input("Digite idade: "))
    so = input("Digite SO [iOS | Android]: ")
    acumuladorIdade += idade

    if so == "iOS":
      contadorIOS += 1
    else:
      contadorAndroid += 1

  mediaIdade = acumuladorIdade / numeroEntrevistados
  print("\nN° iOS: {} N° Android: {} - Média das idades:
{}".format(contadorIOS, contadorAndroid, mediaIdade))

entrevistados()
```

Alguns exemplos:
Operadores matemáticos: +, -, /, *
Operadores relacionais: ==, !=, >, <, >=, <=
Operadores lógicos: and, or, ! (not)

Operadores

Existem operadores lógicos, matemáticos e relacionais.

Fábrica de Software

Primeiro exemplo

PROGRAMA

```
def entrevistados():
  contadorIOS = 0
  contadorAndroid = 0
  acumuladorIdade = 0

  numeroEntrevistados = int(input("Número entrevistados: "))
  for _ in range(numeroEntrevistados):
    idade = int(input("Digite idade: "))
    so = input("Digite SO [iOS | Android]: ")
    acumuladorIdade += idade

    if so == "iOS":
      contadorIOS += 1
    else:
      contadorAndroid += 1

  mediaIdade = acumuladorIdade / numeroEntrevistados
  print("\nN° iOS: {} N° Android: {} - Média das idades:
{}".format(contadorIOS, contadorAndroid, mediaIdade))

entrevistados()
```

A função (método) format() serve para substituir as {} pelos argumentos (valores das variáveis, na ordem em que aparecessem).

O format permite formatar a string de saída.

Fábrica de Software

Modularização

Fonte: https://developerslife.tech/pt/

Fábrica de Software

Modularização - Definição

O Python contém **funções úteis organizadas em bibliotecas** de códigos chamadas **módulos**. Algumas funções fazem parte do **`__buildin__`**, como **`abs()`** e **`round()`**. Entretanto, para usar o **`sqrt()`** é preciso importar o módulo **`math`**.

```
import math

# Agora você pode usar a função sqrt() do módulo math
resultado = math.sqrt(25)

print(resultado)  # Isso imprimirá 5.0, que é a raiz quadrada de 25
```

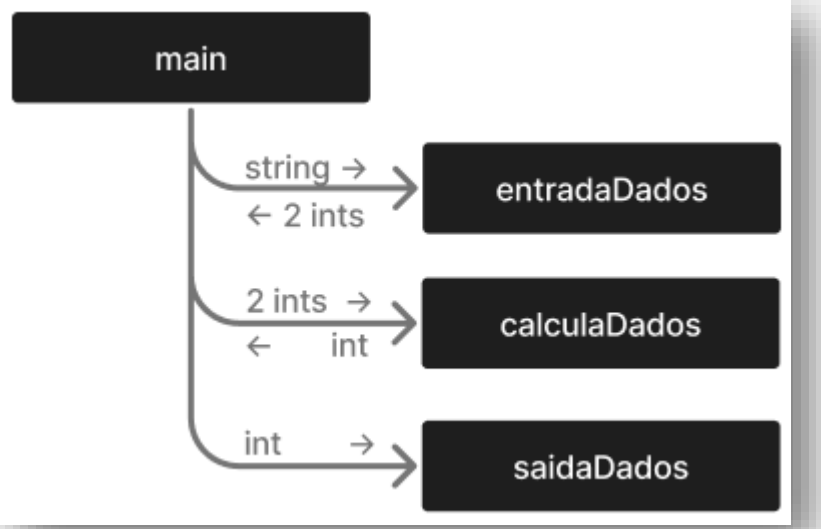
Para uma função específica, pode-se usar: **`from math import sqrt.`** Porém, perde-se a distinção de origem, aumentando o risco de conflitos de nome. Embora útil, a prática comum é importar o módulo inteiro, usando um alias para manter a clareza, como: **`import math as m.`**

Fábrica de Software

Modularização - Definição

Uma **função** é um fragmento de código chamado pelo nome para executar uma tarefa (LAMBERT, 2022).

Pode exigir **argumento/parâmetro(s)** para realizar sua tarefa. Também pode enviar um resultado de **retorno** para o programa que a chamou.



Fábrica de Software

Modularização - Vantagens

1. Simplifica a complexidade de **abstração** de um projeto de software.
2. Dividir o problema em partes **menores e gerenciáveis**.
3. Facilita o **desenvolvimento, teste e manutenção**.
4. Permite a **reutilização** de funcionalidades, economizando tempo e esforço.
5. Elimina **redundâncias**.
6. Serve como **método geral** para uma classe de problemas.

Fábrica de Software

Mais um ...

Segundo exemplo!

STEP 1

Entenda o problema e informações fornecidas

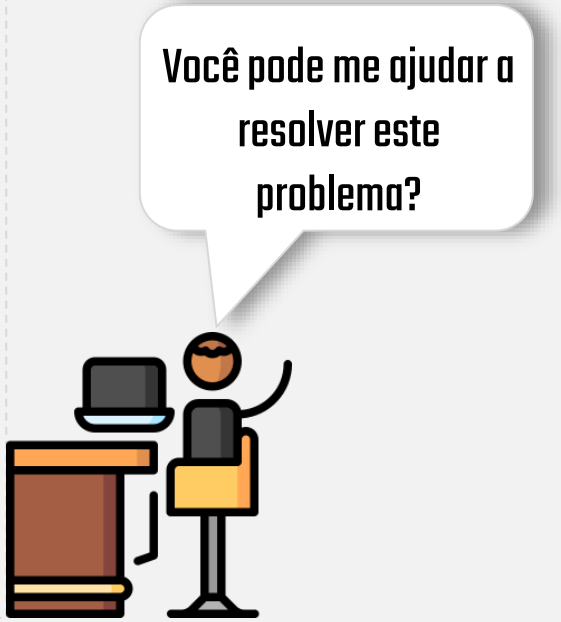
O plano diretor de desenvolvimento urbano de uma cidade determina qual é o percentual de área máximo destinado para garagem em relação à área total do terreno da casa, dependendo da localização desse terreno na cidade:

- Para a zona norte da cidade, o percentual máximo é de 25%.
- Para as zonas leste e oeste da cidade, o percentual máximo é de 30%.
- Para a zona sul, menos povoada, o percentual máximo é de 40%.

Uma empresa de arquitetura está com vários contratos e necessita calcular rapidamente esse percentual, antes de iniciar os projetos.

Faça um programa que recebe as medidas do terreno e da garagem e a zona onde estará localizado o imóvel, calcula o percentual de ocupação da área da garagem em relação ao terreno e emite mensagem sobre o atendimento às regras de ocupação conforme o plano diretor.

Fonte: exemplo do Conteúdo do livro sobre Comando condicional if (estruturas simples e compostas) — Python (página 15).



Fábrica de Software

Exemplo

STEP 2

Identifique objetivo, entradas, saídas e casos de teste

OBJETIVO

Calcular ocupação e verificar o plano diretor.

ENTRADAS

Medidas garagem (largura e profundidade)

Medidas terreno (largura e profundidade)

Localização do terreno (zona)

SAÍDAS

Resultado percentual de ocupação e mensagem conforme ao plano diretor

CASOS DE TESTE

Caso de teste N

Caso de teste 2

Caso de teste 1

Entrada	Saída
Entre com a largura da garagem em metros: 20 Entre com a profundidade da garagem em metros: 20 Entre com a largura do terreno em metros: 40 Entre com a profundidade do terreno em metros: 40 Entre com a zona de localização [Sul=S, Norte=N, Leste=L, Oeste=O]: N	Percentual de ocupação: 25.0 Projeto atende norma de zoneamento do plano diretor

Fábrica de Software

Exemplo

STEP 3

Elabore o algoritmo (textual ou diagrama)

1. Ler valor das medidas do terreno
2. Ler valor das medidas da garagem
3. Calcular percentual de ocupação da garagem
4. Ler zona
5. Verificar regra de ocupação
6. Mostrar mensagem conforme plano direto

Fábrica de Software

Exemplo

Visão geral

STEP 4

Desenvolva o programa e promova melhorias

```python
# Entrada das medidas da garagem e do terreno do imóvel.
def entradaMedidas():
  larguraGaragem = float (input("Entre com a largura da garagem em metros: "))
  profundidadeGaragem = float (input("Entre com a profundidade da garagem em metros: "))
  larguraTerreno = float (input("Entre com a largura do terreno em metros: "))
  profundidadeTerreno = float (input("Entre com a profundidade do terreno em metros: "))
  return larguraGaragem, profundidadeGaragem, larguraTerreno, profundidadeTerreno

# Cálculo do percentual de ocupação da garagem em relação ao terreno.
def calculoOcupacao():
  larguraGaragem, profundidadeGaragem, larguraTerreno, profundidadeTerreno = entradaMedidas()
  areaGaragem = larguraGaragem * profundidadeGaragem # Cálculo da área da garagem.
  areaTerreno = larguraTerreno * profundidadeTerreno # Cálculo da área do terreno.
  percentualOcupacao = (areaGaragem  * 100) / areaTerreno
  print("Percentual de ocupação: {}".format(percentualOcupacao))
  return percentualOcupacao

# Entrada da localização do imóvel.
def zonaLocalizacao():
  zona = input("Entre com a zona de localização [Sul=S, Norte=N, Leste=L, Oeste=O]: ")
  return zona

# Verifica regra de ocupação conforme o plano diretor e apresenta uma mensagem.
def planoDiretor():
  percentual = calculoOcupacao()
  zona = zonaLocalizacao()
  if zona == "N" and percentual <= 25.00:
    print("Projeto atende norma de zoneamento do plano diretor")
  elif zona == "L" or zona == "O" and percentual <= 30.00:
    print("Projeto atende norma de zoneamento do plano diretor")
  elif zona == "S" and percentual <= 40.00:
    print("Projeto atende norma de zoneamento do plano diretor")
  else:
    print("Revisar medidas, projeto não atende norma de zoneamento do plano diretor")

planoDiretor() # Chamada à função principal.
```

Estruturas condicionais

Exemplo

STEP 4

Desenvolva o programa e promova melhorias

```
# Entrada das medidas da garagem e do terreno do imóvel.
def entradaMedidas():
  larguraGaragem = float (input("Entre com a largura da garagem em metros: "))
  profundidadeGaragem = float (input("Entre com a profundidade da garagem em metros: "))
  larguraTerreno = float (input("Entre com a largura do terreno em metros: "))
  profundidadeTerreno = float (input("Entre com a profundidade do terreno em metros: "))
  return larguraGaragem, profundidadeGaragem, larguraTerreno, profundidadeTerreno



# Cálculo do percentual de ocupação da garagem em relação ao terreno.
def calculoOcupacao():
  larguraGaragem, profundidadeGaragem, larguraTerreno, profundidadeTerreno = entradaMedidas()
  areaGaragem = larguraGaragem * profundidadeGaragem # Cálculo da área da garagem.
  areaTerreno = larguraTerreno * profundidadeTerreno # Cálculo da área do terreno.
  percentualOcupacao = (areaGaragem  * 100) / areaTerreno
  print("Percentual de ocupação: {}".format(percentualOcupacao))
  return percentualOcupacao
```

Modular

Estruturas condicionais

Exemplo

cartão

STEP 4

Desenvolva o programa e promova melhorias

```
# Entrada da localização do imóvel.
def zonaLocalizacao():
  zona = input("Entre com a zona de localização [Sul=S, Norte=N, Leste=L, Oeste=O]: ")
  return zona


# Verifica regra de ocupação conforme o plano diretor e apresenta uma mensagem.
def planoDiretor():
  percentual = calculoOcupacao()
  zona = zonaLocalizacao()
  if zona == "N" and percentual <= 25.00:
    print("Projeto atende norma de zoneamento do plano diretor")
  elif zona == "L" or zona == "O" and percentual <= 30.00:
    print("Projeto atende norma de zoneamento do plano diretor")
  elif zona == "S" and percentual <= 40.00:
    print("Projeto atende norma de zoneamento do plano diretor")
  else:
    print("Revisar medidas, projeto não atende norma de zoneamento do plano diretor")


planoDiretor() # Chamada à função principal.
```

Modular

Fábrica de Software

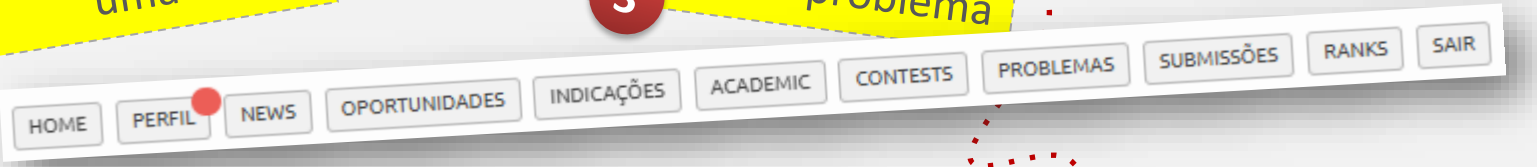
Atividade de Python

Bem-vindos, ao nosso laboratório de programação!

1 Acesse https://www.beecrowd.com.br

2 **Informe** seu login e senha (ou crie uma conta)

3 No menu superior, **selecione problemas -> buscar**, informe o número do problema



4 **Resolva e implemente de forma modular** os seguintes problemas

#			PROBLEMA
1	1006	✔	Média 2
2	1044	✔	Múltiplos
3	1067	✔	Números Ímpares
4	1071	✔	Soma de Impares Consecutivos I
5	1113	✔	Crescente e Decrescente
6	1168	✔	LED
7	1180	✔	Menor e Posição
8	1555	✔	Funções
9	1020	✔	Idade em Dias
10	1017	✔	Gasto de Combustível
11	1001	✔	Extremamente Básico

Caminho para acessar o Beecrowd:

Objetivo:

Explorar a capacidade de compreensão de problemas e de propor solução. Exercitar a competência de abstração. Além de colocar em prática o conhecimento de programação.

Profª Drª Daniela Bagatini

Fábrica de Software

Para o próximo encontro

STEP 3

Aula mediada

Síntese da aprendizagem – conectar conceitos / produção de sentido -

STEP 4

Pós-Aula

Apresentação dos resultados

Entrega pela sala virtual

das Atividades de Python

para os problemas do Beecrowd

Fábrica de Software

O que vamos estudar agora?

Modularização (declaração, parâmetros, bibliotecas) e recursividade

Objetivos de aprendizagem:

Compreender as principais primitivas de programação, organizar os passos de resolução e aplicar na solução de problemas iniciais utilizando recursividade.

Aula preparatória:

Recursividade (conceito e aplicação)

Pré-aula (Estudos individuais):

Investigar e testar diferentes IDEs e escolher uma.

Interpretar, analisar e resolver os problemas propostos no Beecrowd.

Material de referência:

Python Programming Fundamentals - Kent D. Lee (seção 5.9)

Fundamentos de Python: primeiros programas. Lambert, Kenneth A. (p.145 - 150)

Fábrica de Software

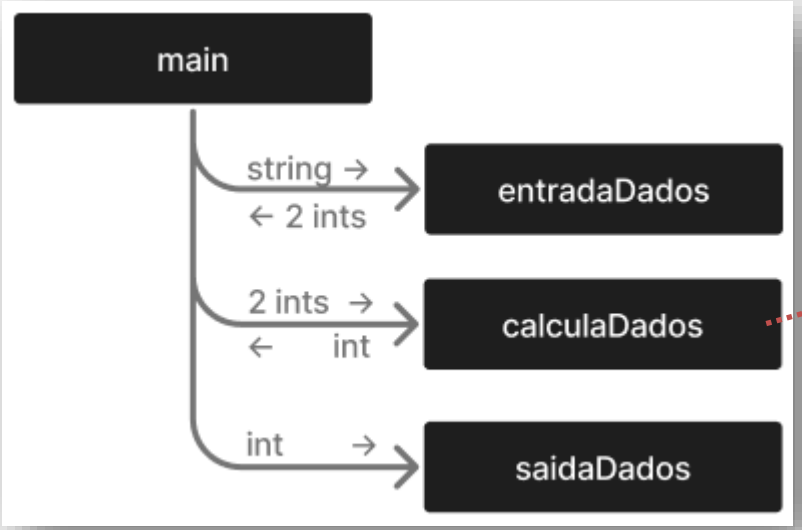
Recursividade

Fonte: https://vidadeprogramador.com.br/2016/01/14/recursividade/

Fábrica de Software

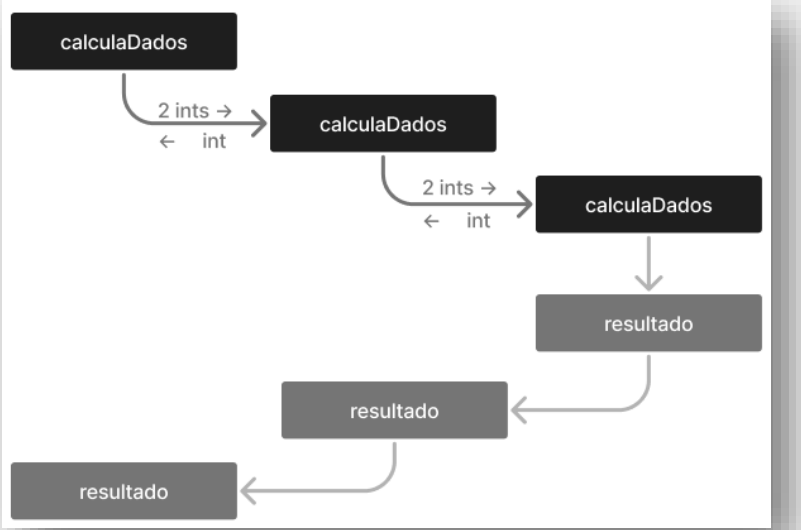
Recursividade

Concepção de **design de cima para baixo**



Concepção de **design recursivo**

Decompor um problema complexo em problemas menores



Fábrica de Software

Recursividade - Definição

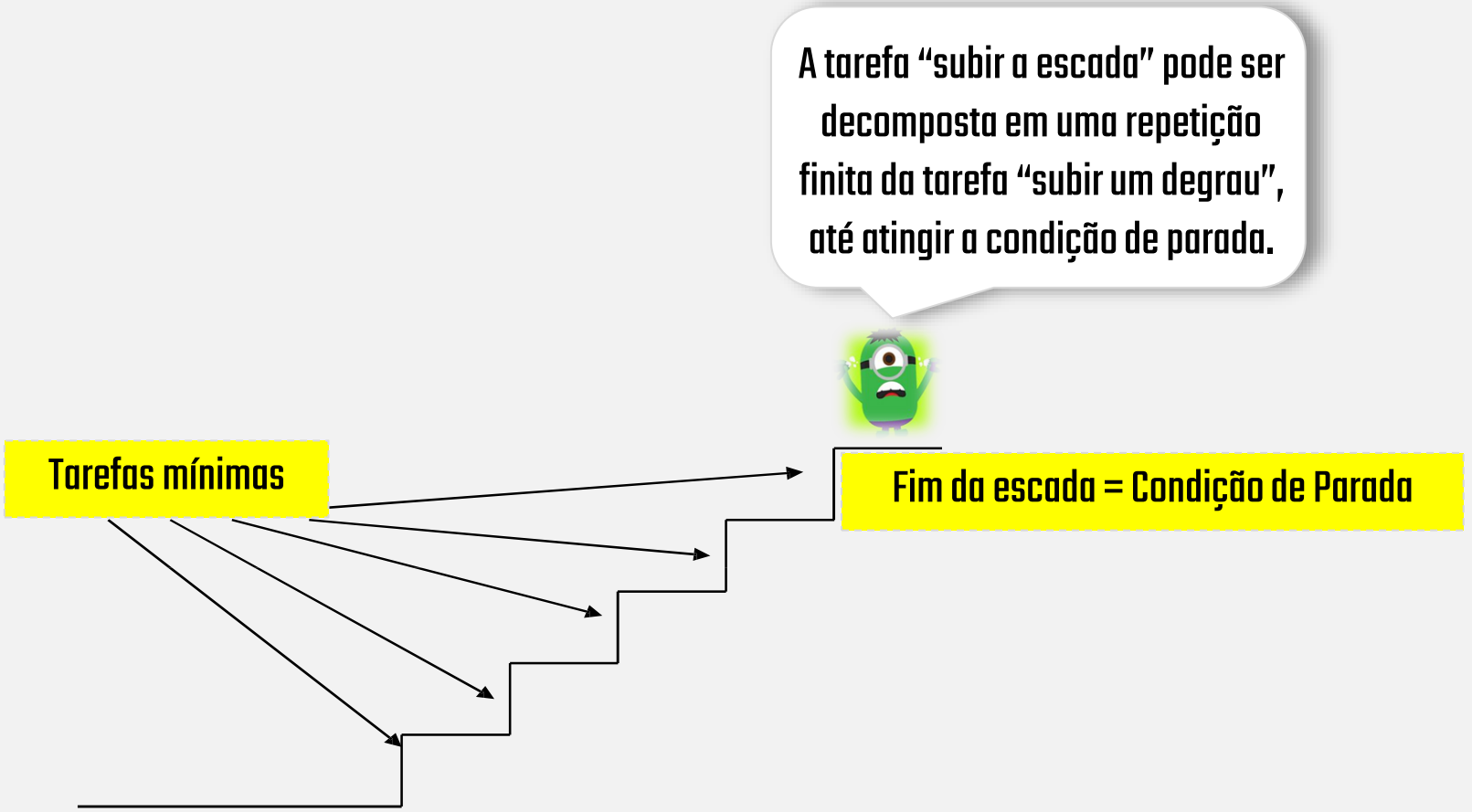
Um objeto é parcialmente definido em termos dele mesmo (PIVA Jr., 2014).

A esse processo dá-se o nome de **recursão**. Portanto, dizemos que a solução de uma tarefa é recursiva quando pode ser definida em termos de si mesma. Ou seja, uma função que chama **ela mesma**.

Fábrica de Software

Recursividade – Tarefa mínima

Na computação esta tarefa não pode ser infinita (ocorreria estouro de memória). Então devemos definir uma **tarefa mínima** e uma **condição de parada**.



Fábrica de Software

Recursividade - Aplicação

Um exemplo de tarefa que pode ser definida como recursiva é o fatorial de um número.

Cálculo do fatorial de um número:

```
0 ! = 1
1 ! = 1 * 0 ! = 1 * 1
2 ! = 2 * 1 ! = 2 * 1 * 1
3 ! = 3 * 2 ! = 3 * 2 * 1 * 1
n ! = n * (n-1)! = n*(n-1)*(n-2)* … * 1
```

Que pode ser formulado como uma função:

$$N\ ! = \begin{cases} 1 & \text{se } N = 0; \\ N * (N-1)! & \text{se } N > 0 \end{cases}$$

Condição de parada (caso base):
0! = 1

Tarefa mínima (caso recursivo):
N! = N*(N -1)!

Fábrica de Software

Recursividade - Exemplo

Podemos usar funções recursivas para resolver problemas definidos recursivamente. Um subprograma é recursivo quando faz **chamadas a si próprio**.

A cada chamada feita à função fatorial, a dimensão do problema (n) diminui, até atingir a condição de parada (n=0).

```python
# Fatorial de um nº é n! = n(n-1)(n-2)(n-3)...
def fatorial(n):
    if n == 0:  # Critério de parada.
        return 1
    return n * fatorial(n - 1)
```

```python
fatorial(4)
```

```
24
```

Condição de parada (caso base) : 0! = 1
Tarefa mínima (caso recursivo) : N! = N*(N -1)!

Regras da Recursividade:

- Um subprograma recursivo deve ter pelo menos uma condição de parada
- Cada chamada recursiva deve ocorrer de modo que sua execução caminhe em direção ao caso base.

Fábrica de Software

Recursividade – Pilha de execução

Exemplo com n = 3

1ª. Chamada

```
Fat= 3 * fat( 2 )
```

2ª. Chamada

```
Fat= 2 * fat (1)
```

3ª. Chamada

```
Fat= 1 * fat (0)
```

Pilha de Execução: para controlar os valores dos parâmetros (n) da função o sistema mantém uma pilha que guarda o endereço e o valor da variável antes da nova chamada ("processos suspensos", que ficam aguardando até poderem se completar).

```python
# Fatorial de um nº é n! = n(n-1)(n-2)(n-3)...
def fatorial(n):
    if n == 0:  # Critério de parada.
        return 1
    return n * fatorial(n - 1)
```

Fábrica de Software

Recursividade – Processo iterativo

```
# A função recursiva recebe dois argumentos: menor e maior. A cada chamada recursiva, o valor de menor é incrementado em 1. A recursão continua até que menor seja maior do que maior, momento em que o menor número é impresso.

def funcaoRecursiva( menor, maior ):
    # Verifica se o menor número é maior do que o maior número
    if menor > maior:
        print( menor )
    else:
        # Continua a recursão com o menor número incrementado
        funcao_recursiva( menor + 1, maior )

# Exemplo de chamada da função recursiva
funcao_recursiva( 3, 8 )
```

```
# A função recursiva recebe dois argumentos: menor e maior. A cada chamada recursiva, o valor de menor é incrementado em 1. A recursão continua até que menor seja maior do que maior, momento em que o menor número é impresso.

def funcaoNaoRecursiva( menor, maior ):
    # Verifica se o menor número é maior do que o maior número
    while menor <= maior:
        menor += 1

    print( menor )

# Exemplo de chamada da função não recursiva
funcao_nao_recursiva( 3, 8 )
```

Fábrica de Software

Recursividade – Processo iterativo

```
#include <stdio.h>

int fat( int n ){
        int x, f = 1;
        for( x =1; x <= n; x++ ){
                f *= x;
        }
        return f;
}

void main(){
        int n, f;
        printf( "Digite o valor para calcular o fatorial: “ );
        scanf( "%d", &n );
        f = fat( n );
        printf( "O fatorial de %d é %d\n", n, f );
}
```

Todo problema resolvido recursivamente também tem uma solução iterativa.

Como seria em Python?

Fábrica de Software

Recursividade – Quando usar

Você **deve** utilizar recursão quando:

- o problema é naturalmente recursivo e a versão recursiva do algoritmo **não gera ineficiência** evidente, se comparada com a versão iterativa;
- o algoritmo se torna **compacto**, sem perder clareza e generalidade;
- é possível **prever que o número de chamadas** (consequentemente, a alocação na pilha) não vão provocar interrupções no processo.

Você **não deve** utilizar recursão quando:

- a solução recursiva causa ineficiência, se comparada com a versão iterativa;
- existe uma única chamada de função recursiva no fim ou no começo da rotina, com a função podendo ser transformada numa interação simples;
- o uso de recursão acarreta número maior de cálculos que a versão interativa;
- parâmetros consideravelmente grandes têm de ser passados por valor;
- não é possível prever o número de chamadas que podem causar sobrecarga da pilha.

Analise a função recursiva `f( )` apresentada em um das questões da prova ENADE*. Após, construa a mesma solução em Python.

QUESTÃO 16

Uma função é denominada recursiva quando ela é chamada novamente dentro de seu corpo. Implementações recursivas tendem a ser menos eficientes, porém facilitam a codificação e seu entendimento.

CELES, W.; CERQUEIRA, R.; RANGEL, J. L. **Introdução a estruturas de dados**. Rio de Janeiro, 2004 (adaptado).

Considere a função recursiva `f()`, a qual foi escrita em linguagem C:

```
int f( int v[], int n) {
    if (n == 0)
        return 0;
    else {
       int s;
       s = f( v, n-1);
      if (v[n-1] > 0) s = s + v[n-1];
        return s;
    }
}
```

Suponha que a função `f()` é acionada com os seguintes parâmetros de entrada:

`f({2,-4,7,0,-1,4},6);`

Nesse caso, o valor de retorno da função `f()` será

*Fonte: Exame Nacional do Ensino Superior

QUESTÃO 16

Uma função é denominada recursiva quando ela é chamada novamente dentro de seu corpo. Implementações recursivas tendem a ser menos eficientes, porém facilitam a codificação e seu entendimento.

CELES, W.; CERQUEIRA, R.; RANGEL, J. L. **Introdução a estruturas de dados**. Rio de Janeiro, 2004 (adaptado).

Considere a função recursiva `f()`, a qual foi escrita em linguagem C:

```
int f( int v[], int n) {
    if (n == 0)
        return 0;
    else {
        int s;
        s = f( v, n-1);
        if (v[n-1] > 0) s = s + v[n-1];
        return s;
    }
}
```

Suponha que a função `f()` é acionada com os seguintes parâmetros de entrada:

```
f({2,-4,7,0,-1,4},6);
```

Nesse caso, o valor de retorno da função `f()` será

	0	1	2	3	4	5
v	2	-4	7	0	-1	4

n = 6

QUESTÃO 16

Uma função é denominada recursiva quando ela é chamada novamente dentro de seu corpo. Implementações recursivas tendem a ser menos eficientes, porém facilitam a codificação e seu entendimento.

CELES, W.; CERQUEIRA, R.; RANGEL, J. L. **Introdução a estruturas de dados**. Rio de Janeiro, 2004 (adaptado).

Considere a função recursiva `f()`, a qual foi escrita em linguagem C:

```
int f( int v[], int n) {
    if (n == 0)
        return 0;
    else {
        int s;
        s = f( v, n-1);
        if (v[n-1] > 0) s = s + v[n-1];
        return s;
    }
}
```

Suponha que a função `f()` é acionada com os seguintes parâmetros de entrada:

```
f({2,-4,7,0,-1,4},6);
```

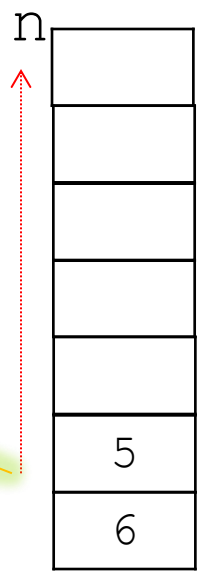
Nesse caso, o valor de retorno da função `f()` será

ENADE 2014
EXAME NACIONAL DE DESEMPENHO DOS ESTUDANTES

	0	1	2	3	4	5
v	2	-4	7	0	-1	4

n = 6

s



ENADE 2014
EXAME NACIONAL DE DESEMPENHO DOS ESTUDANTES

QUESTÃO 16

Uma função é denominada recursiva quando ela é chamada novamente dentro de seu corpo. Implementações recursivas tendem a ser menos eficientes, porém facilitam a codificação e seu entendimento.

CELES, W.; CERQUEIRA, R.; RANGEL, J. L. **Introdução a estruturas de dados**. Rio de Janeiro, 2004 (adaptado).

Considere a função recursiva `f()`, a qual foi escrita em linguagem C:

```
int f( int v[], int n) {
    if (n == 0)
        return 0;
    else {
        int s;
        s = f( v, n-1);
        if (v[n-1] > 0) s = s + v[n-1];
        return s;
    }
}
```

Suponha que a função `f()` é acionada com os seguintes parâmetros de entrada:

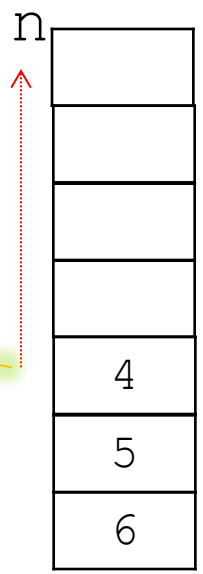
```
f({2,-4,7,0,-1,4},6);
```

Nesse caso, o valor de retorno da função `f()` será

	0	1	2	3	4	5
v	2	-4	7	0	-1	4

n = 6

s



QUESTÃO 16

Uma função é denominada recursiva quando ela é chamada novamente dentro de seu corpo. Implementações recursivas tendem a ser menos eficientes, porém facilitam a codificação e seu entendimento.

CELES, W.; CERQUEIRA, R.; RANGEL, J. L. **Introdução a estruturas de dados**. Rio de Janeiro, 2004 (adaptado).

Considere a função recursiva `f()`, a qual foi escrita em linguagem C:

```
int f( int v[], int n) {
    if (n == 0)
        return 0;
    else {
        int s;
        s = f( v, n-1);
        if (v[n-1] > 0) s = s + v[n-1];
        return s;
    }
}
```

Suponha que a função `f()` é acionada com os seguintes parâmetros de entrada:

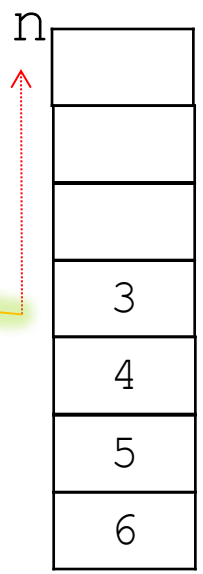
```
f({2,-4,7,0,-1,4},6);
```

Nesse caso, o valor de retorno da função `f()` será

	0	1	2	3	4	5
v	2	-4	7	0	-1	4

n = 6

s



QUESTÃO 16

Uma função é denominada recursiva quando ela é chamada novamente dentro de seu corpo. Implementações recursivas tendem a ser menos eficientes, porém facilitam a codificação e seu entendimento.

CELES, W.; CERQUEIRA, R.; RANGEL, J. L. **Introdução a estruturas de dados**. Rio de Janeiro, 2004 (adaptado).

Considere a função recursiva `f()`, a qual foi escrita em linguagem C:

```
int f( int v[], int n) {
    if (n == 0)
        return 0;
    else {
        int s;
        s = f( v, n-1);
        if (v[n-1] > 0) s = s + v[n-1];
        return s;
    }
}
```

Suponha que a função `f()` é acionada com os seguintes parâmetros de entrada:

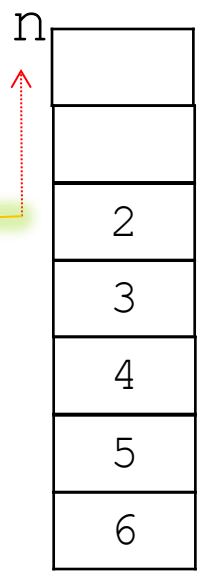
`f({2,-4,7,0,-1,4},6);`

Nesse caso, o valor de retorno da função `f()` será

	0	1	2	3	4	5
v	2	-4	7	0	-1	4

n = 6

s



QUESTÃO 16

Uma função é denominada recursiva quando ela é chamada novamente dentro de seu corpo. Implementações recursivas tendem a ser menos eficientes, porém facilitam a codificação e seu entendimento.

CELES, W.; CERQUEIRA, R.; RANGEL, J. L. **Introdução a estruturas de dados**. Rio de Janeiro, 2004 (adaptado).

Considere a função recursiva `f()`, a qual foi escrita em linguagem C:

```
int f( int v[], int n) {
    if (n == 0)
        return 0;
    else {
        int s;
        s = f( v, n-1);
        if (v[n-1] > 0) s = s + v[n-1];
        return s;
    }
}
```

Suponha que a função `f()` é acionada com os seguintes parâmetros de entrada:

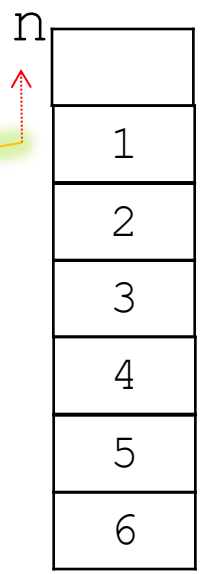
```
f({2,-4,7,0,-1,4},6);
```

Nesse caso, o valor de retorno da função `f()` será

	0	1	2	3	4	5
v	2	-4	7	0	-1	4

n = 6

s



QUESTÃO 16

Uma função é denominada recursiva quando ela é chamada novamente dentro de seu corpo. Implementações recursivas tendem a ser menos eficientes, porém facilitam a codificação e seu entendimento.

CELES, W.; CERQUEIRA, R.; RANGEL, J. L. **Introdução a estruturas de dados**. Rio de Janeiro, 2004 (adaptado).

Considere a função recursiva `f()`, a qual foi escrita em linguagem C:

```
int f( int v[], int n) {
    if (n == 0)
        return 0;
    else {
        int s;
        s = f( v, n-1);
        if (v[n-1] > 0) s = s + v[n-1];
        return s;
    }
}
```

Suponha que a função `f()` é acionada com os seguintes parâmetros de entrada:

```
f({2,-4,7,0,-1,4},6);
```

Nesse caso, o valor de retorno da função `f()` será

	0	1	2	3	4	5
v	2	-4	7	0	-1	4

n = 6

s

n
0
1
2
3
4
5
6

QUESTÃO 16

Uma função é denominada recursiva quando ela é chamada novamente dentro de seu corpo. Implementações recursivas tendem a ser menos eficientes, porém facilitam a codificação e seu entendimento.

CELES, W.; CERQUEIRA, R.; RANGEL, J. L. **Introdução a estruturas de dados**. Rio de Janeiro, 2004 (adaptado).

Considere a função recursiva `f()`, a qual foi escrita em linguagem C:

```
int f( int v[], int n) {
    if (n == 0)
        return 0;
    else {
        int s;
        s = f( v, n-1);
        if (v[n-1] > 0) s = s + v[n-1];
        return s;
    }
}
```

Suponha que a função `f()` é acionada com os seguintes parâmetros de entrada:

```
f({2,-4,7,0,-1,4},6);
```

Nesse caso, o valor de retorno da função `f()` será

	0	1	2	3	4	5
v	2	-4	7	0	-1	4

n = 6

S = 0

n
0
1
2
3
4
5
6

QUESTÃO 16

Uma função é denominada recursiva quando ela é chamada novamente dentro de seu corpo. Implementações recursivas tendem a ser menos eficientes, porém facilitam a codificação e seu entendimento.

CELES, W.; CERQUEIRA, R.; RANGEL, J. L. **Introdução a estruturas de dados**. Rio de Janeiro, 2004 (adaptado).

Considere a função recursiva `f()`, a qual foi escrita em linguagem C:

```
int f( int v[], int n) {
    if (n == 0)
        return 0;
    else {
        int s;
        s = f( v, n-1);
        if (v[n-1] > 0) s = s + v[n-1];
        return s;
    }
}
```

Suponha que a função `f()` é acionada com os seguintes parâmetros de entrada:

```
f({2,-4,7,0,-1,4},6);
```

Nesse caso, o valor de retorno da função `f()` será

	0	1	2	3	4	5
v	2	-4	7	0	-1	4

n = 6

n	1
	2
	3
	4
	5
	6

s=0 (return 0)

s=0+2 s=2

QUESTÃO 16

Uma função é denominada recursiva quando ela é chamada novamente dentro de seu corpo. Implementações recursivas tendem a ser menos eficientes, porém facilitam a codificação e seu entendimento.

CELES, W.; CERQUEIRA, R.; RANGEL, J. L. **Introdução a estruturas de dados**. Rio de Janeiro, 2004 (adaptado).

Considere a função recursiva `f()`, a qual foi escrita em linguagem C:

```
int f( int v[], int n) {
    if (n == 0)
        return 0;
    else {
        int s;
        s = f( v, n-1);
        if (v[n-1] > 0) s = s + v[n-1];
        return s;
    }
}
```

Suponha que a função `f()` é acionada com os seguintes parâmetros de entrada:

```
f({2,-4,7,0,-1,4},6);
```

Nesse caso, o valor de retorno da função `f()` será

	0	1	2	3	4	5
v	2	-4	7	0	-1	4

n = 6

s=0 (return 0)

s=0+2 s=2

n	2
	3
	4
	5
	6

ENADE 2014
EXAME NACIONAL DE DESEMPENHO DOS ESTUDANTES

QUESTÃO 16

Uma função é denominada recursiva quando ela é chamada novamente dentro de seu corpo. Implementações recursivas tendem a ser menos eficientes, porém facilitam a codificação e seu entendimento.

CELES, W.; CERQUEIRA, R.; RANGEL, J. L. **Introdução a estruturas de dados**. Rio de Janeiro, 2004 (adaptado).

Considere a função recursiva `f()`, a qual foi escrita em linguagem C:

```
int f( int v[], int n) {
    if (n == 0)
        return 0;
    else {
        int s;
        s = f( v, n-1);
        if (v[n-1] > 0) s = s + v[n-1];
        return s;
    }
}
```

Suponha que a função `f()` é acionada com os seguintes parâmetros de entrada:

`f({2,-4,7,0,-1,4},6);`

Nesse caso, o valor de retorno da função `f()` será

	0	1	2	3	4	5
v	2	-4	7	0	-1	4

n = 6

n	3
	4
	5
	6

s=0 (return 0)

s=0+2 s=2

s=2+7 s=9

ENADE 2014
EXAME NACIONAL DE DESEMPENHO DOS ESTUDANTES

QUESTÃO 16

Uma função é denominada recursiva quando ela é chamada novamente dentro de seu corpo. Implementações recursivas tendem a ser menos eficientes, porém facilitam a codificação e seu entendimento.

CELES, W.; CERQUEIRA, R.; RANGEL, J. L. **Introdução a estruturas de dados**. Rio de Janeiro, 2004 (adaptado).

Considere a função recursiva `f()`, a qual foi escrita em linguagem C:

```
int f( int v[], int n) {
    if (n == 0)
        return 0;
    else {
        int s;
        s = f( v, n-1);
        if (v[n-1] > 0) s = s + v[n-1];
        return s;
    }
}
```

Suponha que a função `f()` é acionada com os seguintes parâmetros de entrada:

```
f({2,-4,7,0,-1,4},6);
```

Nesse caso, o valor de retorno da função `f()` será

	0	1	2	3	4	5
v	2	-4	7	0	-1	4

n = 6

n	4
	5
	6

s=0 (return 0)

s=0+2 s=2

s=2+7 s=9

ENADE 2014
EXAME NACIONAL DE DESEMPENHO DOS ESTUDANTES

QUESTÃO 16

Uma função é denominada recursiva quando ela é chamada novamente dentro de seu corpo. Implementações recursivas tendem a ser menos eficientes, porém facilitam a codificação e seu entendimento.

CELES, W.; CERQUEIRA, R.; RANGEL, J. L. **Introdução a estruturas de dados**. Rio de Janeiro, 2004 (adaptado).

Considere a função recursiva `f()`, a qual foi escrita em linguagem C:

```
int f( int v[], int n) {
    if (n == 0)
        return 0;
    else {
        int s;
        s = f( v, n-1);
        if (v[n-1] > 0) s = s + v[n-1];
        return s;
    }
}
```

Suponha que a função `f()` é acionada com os seguintes parâmetros de entrada:

`f({2,-4,7,0,-1,4},6);`

Nesse caso, o valor de retorno da função `f()` será

	0	1	2	3	4	5
v	2	-4	7	0	-1	4

n = 6

s=0 (return 0)

s=0+2 s=2

s=2+7 s=9

n	5
	6

ENADE 2014
EXAME NACIONAL DE DESEMPENHO DOS ESTUDANTES

QUESTÃO 16

Uma função é denominada recursiva quando ela é chamada novamente dentro de seu corpo. Implementações recursivas tendem a ser menos eficientes, porém facilitam a codificação e seu entendimento.

CELES, W.; CERQUEIRA, R.; RANGEL, J. L. **Introdução a estruturas de dados**. Rio de Janeiro, 2004 (adaptado).

Considere a função recursiva `f()`, a qual foi escrita em linguagem C:

```
int f( int v[], int n) {
    if (n == 0)
        return 0;
    else {
        int s;
        s = f( v, n-1);
        if (v[n-1] > 0) s = s + v[n-1];
        return s;
    }
}
```

Suponha que a função `f()` é acionada com os seguintes parâmetros de entrada:

```
f({2,-4,7,0,-1,4},6);
```

Nesse caso, o valor de retorno da função `f()` será

	0	1	2	3	4	5
v	2	-4	7	0	-1	4

n = 6

n 6

s=0 (return 0)

s=0+2 s=2

s=2+7 s=9

s=9+4 **s=13**

QUESTÃO 16

Uma função é denominada recursiva quando ela é chamada novamente dentro de seu corpo. Implementações recursivas tendem a ser menos eficientes, porém facilitam a codificação e seu entendimento.

CELES, W.; CERQUEIRA, R.; RANGEL, J. L. **Introdução a estruturas de dados**. Rio de Janeiro, 2004 (adaptado).

Considere a função recursiva `f()`, a qual foi escrita em linguagem C:

```
int f( int v[], int n) {
    if (n == 0)
        return 0;
    else {
        int s;
        s = f( v, n-1);
        if (v[n-1] > 0) s = s + v[n-1];
        return s;
    }
}
```

Suponha que a função `f()` é acionada com os seguintes parâmetros de entrada:

```
f({2,-4,7,0,-1,4},6);
```

Nesse caso, o valor de retorno da função `f()` será

- **A** 8.
- **B** 10.
- **C** 13.
- **D** 15.
- **E** 18.

	0	1	2	3	4	5
v	2	-4	7	0	-1	4

n = 6

s=0 (return 0)

s=0+2 s=2

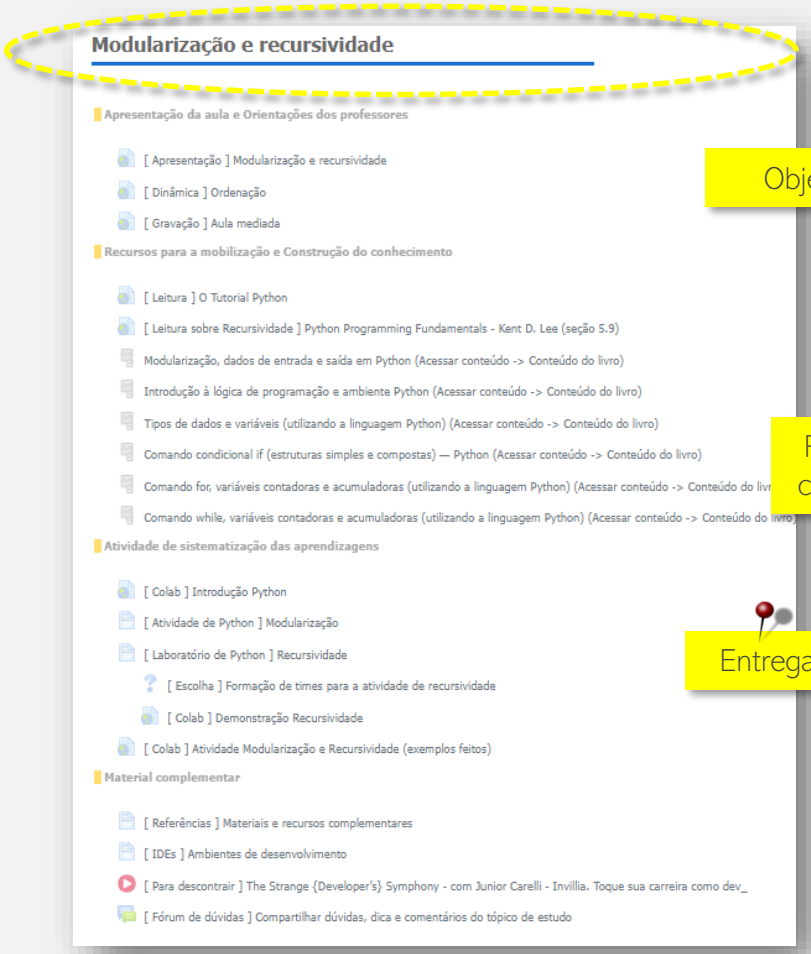
s=2+7 s=9

n

s=9+4 **s=13**

Fábrica de Software

O que trabalhamos nesta aula?



Modularização e recursividade

Apresentação da aula e Orientações dos professores

- [Apresentação] Modularização e recursividade
- [Dinâmica] Ordenação
- [Gravação] Aula mediada

Recursos para a mobilização e Construção do conhecimento

- [Leitura] O Tutorial Python
- [Leitura sobre Recursividade] Python Programming Fundamentals - Kent D. Lee (seção 5.9)
- Modularização, dados de entrada e saída em Python (Acessar conteúdo -> Conteúdo do livro)
- Introdução à lógica de programação e ambiente Python (Acessar conteúdo -> Conteúdo do livro)
- Tipos de dados e variáveis (utilizando a linguagem Python) (Acessar conteúdo -> Conteúdo do livro)
- Comando condicional if (estruturas simples e compostas) — Python (Acessar conteúdo -> Conteúdo do livro)
- Comando for, variáveis contadoras e acumuladoras (utilizando a linguagem Python) (Acessar conteúdo -> Conteúdo do livr
- Comando while, variáveis contadoras e acumuladoras (utilizando a linguagem Python) (Acessar conteúdo -> Conteúdo do livro)

Atividade de sistematização das aprendizagens

- [Colab] Introdução Python
- [Atividade de Python] Modularização
- [Laboratório de Python] Recursividade
  - [Escolha] Formação de times para a atividade de recursividade
  - [Colab] Demonstração Recursividade
- [Colab] Atividade Modularização e Recursividade (exemplos feitos)

Material complementar

- [Referências] Materiais e recursos complementares
- [IDEs] Ambientes de desenvolvimento
- [Para descontrair] The Strange {Developer's} Symphony - com Junior Carelli - Invillia. Toque sua carreira como dev_
- [Fórum de dúvidas] Compartilhar dúvidas, dica e comentários do tópico de estudo

Objetivos

Principais conteúdos

Entrega

Trabalhamos assuntos como:

- modularização e recursividade:
  - sintaxe,
  - parâmetros,
  - passagem de parâmetros,
  - escopo de variável,
  - recursão,
  - tarefa mínima.

Competências desenvolvidas:

- Raciocínio lógico.
- Compreensão das principais primitivas de programação.
- Organização dos passos de resolução e aplicar na solução de problemas iniciais.
- Aplicação em Python.

Fábrica de Software

Atividade de Python

Bem-vindos, ao nosso laboratório de programação!

1 Acesse https://www.beecrowd.com.br

2 **Informe** seu login e senha (ou crie uma conta)

3 No menu superior, **selecione problemas -> buscar**, informe o número do problema

4 **Resolva e implemente de forma modular** os seguintes problemas: **1028 ou 1151**

Caminho para acessar o Beecrowd:

Objetivo:

Explorar a capacidade de compreensão de problemas e de propor solução.
Exercitar a competência de abstração.
Além de colocar em prática o conhecimento de programação.

Fábrica de Software

Laboratório de Python - Recursividade

- Após formar seu **time**, acesse o site Beecrowd e **resolva** os problemas 1028 ou 1151 utilizando recursividade.
- Lembre-se de estabelecer a condição de parada e o caso recursivo (**tarefa mínima**).

URI Online Judge | 1151

Fibonacci Fácil

Adaptado por Neilor Tonin, URI Brasil

Timelimit: 1

A seguinte sequência de números 0 1 1 2 3 5 8 13 21... é conhecida como série de Fibonacci. Nessa sequência, cada número, depois dos 2 primeiros, é igual à soma dos 2 anteriores. Escreva um algoritmo que leia um inteiro N (N < 46) e mostre os N primeiros números dessa série.

https://www.urionlinejudge.com.br/judge/pt/problems/view/1151

URI Online Judge | 1028

Figurinhas

Por Neilor Tonin, URI Brasil

Timelimit: 1

Ricardo e Vicente são aficionados por figurinhas. Nas horas vagas, eles arrumam um jeito de jogar um "bafo" ou algum outro jogo que envolva tais figurinhas. Ambos também têm o hábito de trocarem as figuras repetidas com seus amigos e certo dia pensaram em uma brincadeira diferente. Chamaram todos os amigos e propuseram o seguinte: com as figurinhas em mãos, cada um tentava fazer uma troca com o amigo que estava mais perto seguindo a seguinte regra: cada um contava quantas figurinhas tinha. Em

https://www.urionlinejudge.com.br/judge/pt/problems/view/1028

Fábrica de Software

Para este encontro

STEP 4

Pós-Aula

Apresentação dos resultados

Entrega pela sala virtual do Laboratório de Python desenvolvido pelo time

Fábrica de Software

Referências

Sobre Desafios

- https://www.beecrowd.com.br/

Sobre Python

- https://www.python.org/
- https://docs.python.org/3/

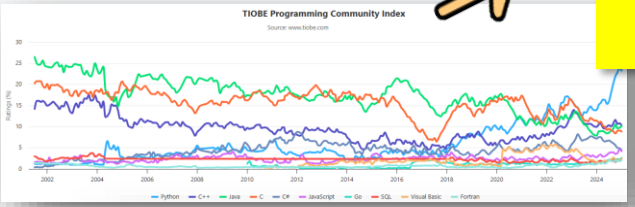
Simples e intuitivo, foco nos objetivos, multiplataforma, tipagem dinâmica e forte, multiparadigma, elegante, variedade de IDEs, integração de testes e ferramentas de refatoração, código aberto e gratuito, vasto material.

Código Fonte TV. Perspectivas para Python em 2024. Disponível em: <https://www.youtube.com/watch?v=4kd1iGRuxXM>. Acesso em: Fevereiro, 2024.

Puga, Sandra. Lógica de programação e estrutura de dados. Capítulo 9 - Recursividade. São Paulo: Pearson Education do Brasil, 2016. (Biblioteca Virtual)

Lee, Kent D. Python Programming Fundamentals (seção 5.9) (ver na sala virtual).

Conheça as linguagens mais utilizadas... Veja que C e Python estão entre as primeiras.



Jan 2025	Jan 2024	Change	Programming Language	Ratings	Change
1	1		Python	23.28%	+9.32%
2	3	^	C++	10.29%	+0.33%
3	4	^	Java	10.15%	+2.28%
4	2	v	C	8.86%	-2.59%
5	5		C#	4.45%	-2.71%

Fonte: https://www.tiobe.com/tiobe-index/